F1 · DAY3 · WEEK1 BLOCS 1 ET 2 · 10H00 – 13H00 PARCOURS AI/ML ENGINEER

Fonctions et fichiers

Bloc 1 : créer des fonctions, comprendre paramètres et valeurs de retour, en enchaîner plusieurs, distinguer print et return. Bloc 2 : lire et écrire des fichiers texte avec `with open`, puis stocker des données au format JSON. Chaque résultat est énoncé rigoureusement, puis traduit « en clair ».

1. Définir et appeler une fonction
2. return, print et None
3. Portée des variables
4. Composer plusieurs fonctions
5. Exercices du bloc 1
6. Ouvrir un fichier
7. Lire un fichier
8. Écrire un fichier
9. Le format JSON
10. Exercices du bloc 2
11. À savoir par cœur

Pourquoi ces deux blocs vont ensemble

Un pipeline d'IA, c'est une suite de fonctions qui se passent des données : **lire** un fichier, **nettoyer**, **calculer**, **sauvegarder** le résultat. Les fonctions sont les machines, les fichiers sont l'entrée et la sortie de la chaîne.

1. Définir et appeler une fonction

Définition 1.1 *(fonction)*

Une **fonction** est un bloc de code nommé, qu'on écrit une fois et qu'on exécute autant de fois qu'on veut. Elle se définit ainsi :

```
def nom_de_la_fonction(parametre1, parametre2):
    instructions          # le CORPS, décalé de 4 espaces
    return resultat
```

En clair : une machine. `def` la fabrique, l'appel l'allume, `return` fait sortir le résultat.

Définition 1.2 *(paramètre, argument, appel)*

Un **paramètre** est un nom écrit entre les parenthèses de `def` : c'est une case vide. Un **argument** est la valeur réelle placée dans cette case au moment de l'**appel** `nom(argument)` .

```
def moyenne(a, b):              # a et b : paramètres
    return (a + b) / 2

m = moyenne(12, 16)             # 12 et 16 : arguments → m vaut 14.0
```

Proposition 1.3 *(définir n'exécute rien)*

L'instruction `def` enregistre la fonction mais n'exécute pas son corps. Le corps ne s'exécute qu'au moment d'un appel.

Définition 1.4 *(arguments positionnels et nommés, valeur par défaut)*

```
def saluer(prenom, formule="Bonjour"):       # formule a une valeur PAR DÉF
    return f"{formule} {prenom}"

saluer("Lina")                      # 'Bonjour Lina'      positionnel
saluer("Lina", "Salut")             # 'Salut Lina'        positionnel
saluer(prenom="Lina", formule="Yo")# 'Yo Lina'            nommés
```

En clair : un argument **positionnel** est rangé selon sa place ; un argument **nommé** dit explicitement dans quelle case il va. La valeur par défaut est le réglage d'usine.

---

2. `return` , `print` et `None`

Proposition 2.1 *(effet de return)*

Quand Python exécute `return valeur` , la fonction **s'arrête immédiatement** et l'appel est remplacé par `valeur` . Les lignes situées après le `return` ne sont jamais exécutées.

Proposition 2.2 *(valeur None)*

Une fonction qui se termine sans exécuter de `return` renvoie la valeur spéciale `None`, qui signifie « rien ».

Corollaire 2.3 *(print ≠ return)*

`print` affiche une valeur à l'écran mais ne la renvoie pas. Une fonction qui contient seulement `print` renvoie donc `None`.

```
def double_print(n):
    print(n * 2)

r = double_print(5)     # affiche 10
print(r)                # None   ← c'est la question 4 du QCM
```

	`PRINT(X)`	`RETURN X`
Image	montrer le plat 👀	donner le plat 🎁
La valeur est-elle réutilisable ?	non	oui : on peut la ranger, la tester, la passer à une autre fonction
Arrête la fonction ?	non	oui (Proposition 2.1)

Remarque 2.4 *(règle de conception)*

Une fonction de **calcul** utilise `return`. On réserve `print` à une seule fonction, celle qui **affiche** le résultat final. C'est ce qui rend les fonctions testables avec `assert`.

3. Portée des variables

Définition 3.1 *(variable locale)*

Une variable créée dans le corps d'une fonction (paramètres compris) est **locale** : elle n'existe que pendant l'appel et n'est pas visible à l'extérieur.

```
def calculer():
    total = 50
    return total

calculer()
print(total)            # NameError: name 'total' is not defined
resultat = calculer()   # ✓ on récupère la valeur grâce à return
```

> **Corollaire 3.2**
>
> Le seul moyen propre de faire sortir une valeur d'une fonction est `return`, et le seul moyen propre d'y faire entrer une valeur est un paramètre.
>
> **En clair :** une fonction est une pièce fermée avec une porte d'entrée (les paramètres) et une porte de sortie (`return`).

---

4. Composer plusieurs fonctions

> **Méthode 4.1** *(une fonction = une tâche)*
>
> On découpe un problème en tâches simples, une fonction par tâche, avec un **verbe** comme nom. La sortie (`return`) de l'une devient l'entrée (argument) de la suivante.

```python
def nettoyer(notes):
    """Garde seulement les notes entre 0 et 20."""
    return [n for n in notes if 0 <= n <= 20]

def moyenne(notes):
    return sum(notes) / len(notes)

def mention(m):
    if m >= 16:
        return "Très bien"
    elif m >= 10:
        return "Admis"
    else:
        return "Ajourné"

brutes = [12, 25, 15, -3, 9]
propres = nettoyer(brutes)          # [12, 15, 9]
m = moyenne(propres)                # 12.0
print(mention(m))                   # Admis

# Même chose en une ligne : les appels s'emboîtent, de l'intérieur vers l'extéri
print(mention(moyenne(nettoyer(brutes))))
```

Dans le métier

C'est la forme exacte d'un pipeline ML : `evaluer(entrainer(preparer(charger(chemin))))`. ExamPredict est construit ainsi : `load_documents` → `analyze_documents` → `calculate_priorities` → `display_results`.

À dire

« Une bonne fonction fait une seule chose. Elle reçoit par ses paramètres, elle donne par return. Et on les branche les unes derrière les autres, comme une chaîne d'usine. C'est ça, un pipeline d'IA. »

5. Exercices du bloc 1

Fichier : `WEEK1/DAY3/exercices/b1_fonctions.py`. Écris, lance, puis compare avec le corrigé.

Exercice 1.A · Prédire ★

```
def f(x):
    return x + 1
    print("fin")

print(f(f(3)))
```

Corrigé. `5`. f(3) = 4, puis f(4) = 5. « fin » n'est jamais affiché, car il est situé après le `return` (Proposition 2.1).

Exercice 1.B · Écrire `maximum(a, b)` sans utiliser max ★

```
def maximum(a, b):
    if a >= b:
        return a
    return b            # on n'arrive ici que si a < b (Proposition 2.1)

assert maximum(3, 7) == 7
assert maximum(9, 2) == 9
```

Exercice 1.C · Valeur par défaut ★★

Écrire `sur_100(note, bareme=20)` qui convertit une note sur `bareme` en note sur 100.

```
def sur_100(note, bareme=20):
    return note / bareme * 100

assert sur_100(15) == 75
assert sur_100(8, bareme=10) == 80
```

Exercice 1.D · Composer trois fonctions ★★

Avec `nettoyer`, `moyenne` et `mention` (section 4), afficher la mention de `[18, 17, 40, 15]`.

Corrigé. `nettoyer` donne `[18, 17, 15]`, la moyenne vaut 16.67, donc « Très bien ».

6. Ouvrir un fichier

Définition 6.1 *(fichier texte, encodage)*

Un **fichier texte** est une suite de lignes de caractères, rangée sur le disque à un **chemin** donné. L'**encodage** est la règle qui transforme les caractères en octets. On utilise toujours `utf-8`, qui gère les accents.

Définition 6.2 *(open et modes)*

`open(chemin, mode, encoding="utf-8")` ouvre un fichier et renvoie un objet fichier.

MODE	SENS	SI LE FICHIER EXISTE	S'IL N'EXISTE PAS
`"r"`	lire (par défaut)	on le lit	`FileNotFoundError`
`"w"`	écrire	**effacé**, puis réécrit	créé
`"a"`	ajouter	on écrit à la fin	créé

Proposition 6.3 *(with)*

Dans `with open(...) as f:`, le fichier est **automatiquement fermé** à la sortie du bloc décalé, même si une erreur survient pendant la lecture.

```
with open("notes.txt", "r", encoding="utf-8") as f:
    contenu = f.read()
# ici, f est déjà fermé
```

En clair : `with` est une porte qui se referme toute seule derrière toi.

7. Lire un fichier

INSTRUCTION	RENVOIE
`f.read()`	tout le fichier, en une seule chaîne
`f.readlines()`	une liste de lignes, chacune terminée par `"\n"`
`for ligne in f:`	les lignes une par une (idéal pour un gros fichier)

Proposition 7.1 *(une ligne lue est du texte)*

Chaque ligne lue est un `str` qui se termine par le caractère invisible `"\n"` (retour à la ligne). Pour calculer, il faut donc la **nettoyer** avec `.strip()`, puis la **convertir** avec `int()` ou `float()`.

Méthode 7.2 *(lire une liste de nombres)*

```
def lire_nombres(chemin):
    nombres = []
    with open(chemin, encoding="utf-8") as f:
        for ligne in f:
            ligne = ligne.strip()        # "14\n" → "14"
            if ligne != "":              # on saute les lignes vides
                nombres.append(float(ligne))
    return nombres
```

Méthode 7.3 *(découper une ligne : split)*

```
ligne = "Lina;14"
morceaux = ligne.split(";")      # ['Lina', '14']
nom = morceaux[0]                # 'Lina'
note = float(morceaux[1])        # 14.0
```

`split(";")` coupe le texte à chaque `;`. C'est le principe des fichiers CSV, le format de base des datasets.

8. Écrire un fichier

Méthode 8.1

```
with open("rapport.txt", "w", encoding="utf-8") as f:
    f.write("Moyenne : 12.5\n")       # write n'ajoute PAS de retour à la l
    f.write(f"Admis : {4} / {6}\n")
```

Piège 8.2

`"w"` efface le fichier existant sans prévenir. Pour garder l'historique (un journal d'entraînement, par exemple), on ouvre en `"a"`. Et `f.write()` n'accepte que du texte : `f.write(12)` provoque une `TypeError`, il faut `f.write(str(12))`.

9. Le format JSON

Définition 9.1 *(JSON)*

JSON (*JavaScript Object Notation*) est un format **texte** pour stocker des données structurées. Il ressemble presque exactement aux dictionnaires et aux listes de Python.

```
{
  "nom": "Lina",
  "notes": [14, 16.5, 12],
  "admis": true,
  "option": null
}
```

PYTHON	JSON
`dict`	objet `{ }`
`list`	tableau `[ ]`
`str`	chaîne, toujours entre **guillemets doubles** `" "`
`int`, `float`	nombre
`True` / `False`	`true` / `false` (minuscules)
`None`	`null`

Méthode 9.2 *(sauvegarder et recharger)*

```
import json

rapport = {"moyenne": 12.67, "admis": 4, "rejets": ["Yanis;abc"]}

# ÉCRIRE : dict Python → fichier JSON
with open("rapport.json", "w", encoding="utf-8") as f:
    json.dump(rapport, f, indent=2, ensure_ascii=False)

# LIRE : fichier JSON → dict Python
with open("rapport.json", encoding="utf-8") as f:
    donnees = json.load(f)

print(donnees["moyenne"])     # 12.67
```

`indent=2` rend le fichier lisible par un humain. `ensure_ascii=False` garde les accents tels quels.

Piège 9.3 *(dump/load ou dumps/loads)*

`json.dump` et `json.load` travaillent avec un **fichier**. `json.dumps` et `json.loads` (avec un **s** pour *string*) travaillent avec une **chaîne** de texte.

> **Dans le métier**
>
> JSON est partout en IA : les réglages d'un entraînement (`config.json`), les métriques sauvegardées après chaque essai, et les réponses des API, y compris celle de Claude. Le futur ExamPredict renverra ses prédictions en JSON au site web.

---

10. Exercices du bloc 2

Fichier : `WEEK1/DAY3/exercices/b2_fichiers.py` , avec le fichier de données `notes.txt` .

Exercice 2.A · Lire et compter ★

Lire `notes.txt` (un nombre par ligne) et afficher le nombre de lignes non vides.

```
with open("notes.txt", encoding="utf-8") as f:
    lignes = [l.strip() for l in f if l.strip() != ""]
print(len(lignes))
```

Exercice 2.B · Lire, calculer, écrire ★★

Avec `lire_nombres` (Méthode 7.2), calculer la moyenne et l'écrire dans `moyenne.txt` .

```
notes = lire_nombres("notes.txt")
m = sum(notes) / len(notes)
with open("moyenne.txt", "w", encoding="utf-8") as f:
    f.write(f"Moyenne : {m:.2f}\n")      # :.2f = 2 chiffres après la virgu
```

Exercice 2.C · JSON aller-retour ★★

Sauvegarder `{"eleve": "Lina", "notes": [14, 16]}` dans `eleve.json`, le relire, puis afficher la moyenne des notes relues.

```
import json
with open("eleve.json", "w", encoding="utf-8") as f:
    json.dump({"eleve": "Lina", "notes": [14, 16]}, f, indent=2)
with open("eleve.json", encoding="utf-8") as f:
    d = json.load(f)
print(sum(d["notes"]) / len(d["notes"]))     # 15.0
```

Exercice 2.D · Prédire ★★★

```
with open("t.txt", "w", encoding="utf-8") as f:
    f.write("A")
with open("t.txt", "w", encoding="utf-8") as f:
    f.write("B")
with open("t.txt", "a", encoding="utf-8") as f:
    f.write("C")
print(open("t.txt", encoding="utf-8").read())
```

Corrigé. `BC`. Le 2e `"w"` efface « A » et écrit « B », puis `"a"` ajoute « C » à la fin.

11. À savoir par cœur

1. `def` fabrique, l'appel exécute, `return` donne et arrête la fonction.
2. Sans `return`, une fonction renvoie `None`. `print` montre, il ne donne pas.
3. Le paramètre est la case, l'argument est la valeur. Une valeur par défaut, c'est le réglage d'usine.
4. Une variable locale meurt à la fin de l'appel : on entre par les paramètres, on sort par `return`.
5. Une fonction = une tâche = un verbe. Les sorties des unes deviennent les entrées des autres.
6. `with open(chemin, mode, encoding="utf-8") as f:` referme le fichier tout seul.

7	`"r"` lit, `"w"` efface puis écrit, `"a"` ajoute.
8	Une ligne lue est du texte : `.strip()`, puis `.split(";")`, puis `float()`.
9	`f.write()` n'écrit que du texte et n'ajoute pas `\n` tout seul.
10	JSON : `json.dump(obj, f)` pour écrire, `json.load(f)` pour lire. `true` / `null` en JSON correspondent à `True` / `None` en Python.
